

# NEUROLEARN

Sistema Operacional de Aprendizagem

## Documentação Técnica

Arquitetura, stack, padrões e engenharia

Versão 2.0 · 10 de junho de 2026

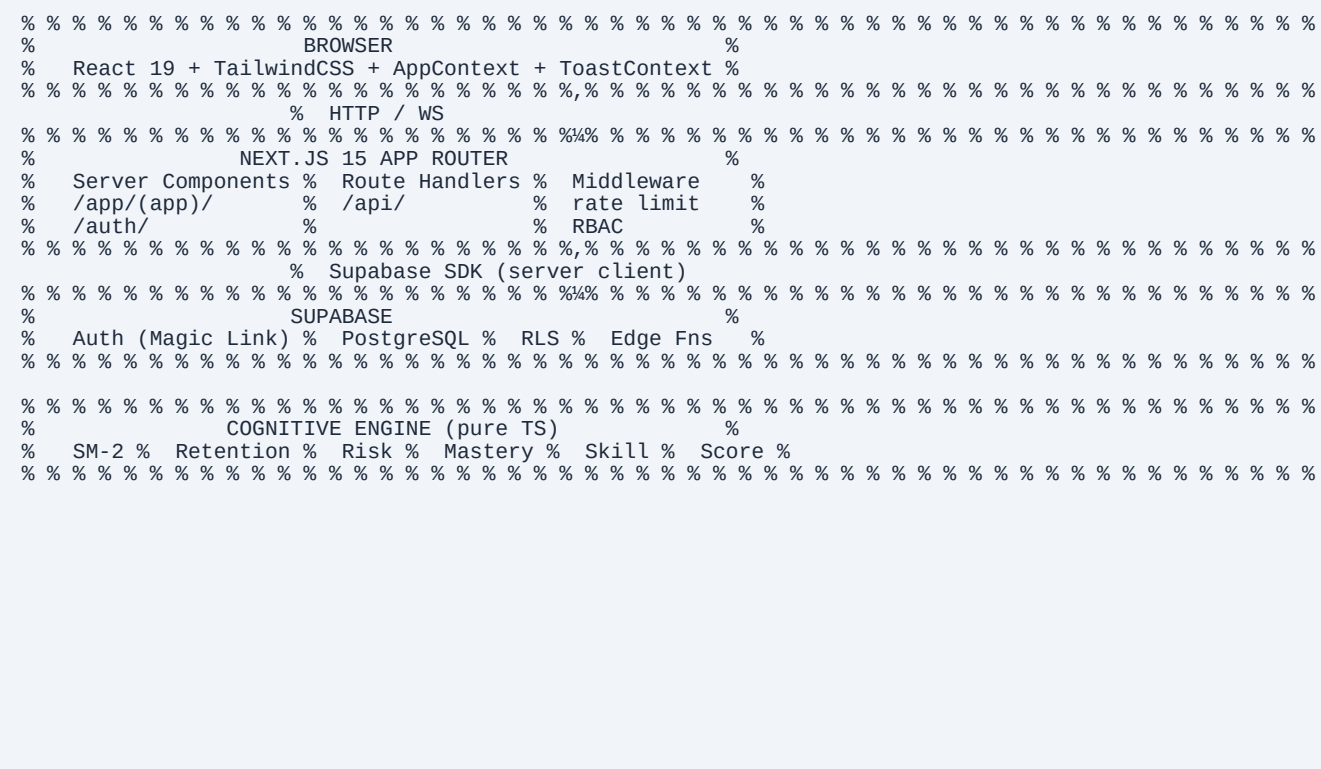
---

# NeuroLearn — Documentação Técnica

Versão: 2.1  
Última atualização: 2026-06-08  
Público: Engenheiros, contribuidores, IA coding agents

## Visão Arquitetural

O NeuroLearn é uma aplicação web full-stack com arquitetura server-first (Next.js App Router), backend gerenciado (Supabase) e uma camada de domínio cognitivo pura em TypeScript.



## Stack Tecnológica

### Frontend

| Tecnologia      | Versão | Papel                                 |
|-----------------|--------|---------------------------------------|
| Next.js         | 15.3.3 | Framework full-stack (App Router)     |
| React           | 19     | UI library                            |
| TypeScript      | 5.7    | Tipagem estática (strict mode)        |
| TailwindCSS     | 3.4    | Estilização utility-first             |
| React Hook Form | latest | Gerenciamento de formulários          |
| Zod             | 4.4.3  | Validação e type inference de schemas |

## Backend / Infraestrutura

| Tecnologia         | Versão | Papel                                |
|--------------------|--------|--------------------------------------|
| Supabase           | latest | Auth + PostgreSQL + Edge Functions   |
| Next.js API Routes | 15     | Endpoints server-side                |
| OpenAI SDK         | latest | Geração de conteúdo IA               |
| Anthropic SDK      | latest | Geração de conteúdo IA (alternativa) |

## Testes

| Tecnologia                | Versão | Papel                           |
|---------------------------|--------|---------------------------------|
| Vitest                    | 4.1.8  | Testes unitários                |
| Playwright                | 1.60.0 | Testes E2E (browser)            |
| @testing-library/react    | latest | Render de componentes em Vitest |
| @testing-library/jest-dom | latest | Matchers DOM customizados       |
| jsdom                     | latest | Ambiente DOM simulado no Vitest |

## Observabilidade

| Tecnologia     | Versão | Papel                                               |
|----------------|--------|-----------------------------------------------------|
| @sentry/nextjs | 10.x   | Error tracking, session replay, performance tracing |
| posthog-js     | latest | Product analytics, page views, feature flags        |

## Utilitários

| Tecnologia           | Papel                      |
|----------------------|----------------------------|
| docx                 | Geração de documentos Word |
| pdfkit               | Geração de PDFs            |
| @vitejs/plugin-react | JSX transform no Vitest    |

## Estrutura de Pastas

```

.
% % % src/
% % % % app/ # Rotas Next.js (App Router)
% % % % % (app)/ # Grupo de rotas autenticadas
% % % % % % layout.tsx # Verifica sessão + carrega AppContext
% % % % % % dashboard/
% % % % % % library/
% % % % % % focus/
% % % % % % % [contentId]/ # Rota dinâmica para sessão de foco
% % % % % % review/
% % % % % % % active/
% % % % % % % skills/
% % % % % % % help/
% % % % % % auth/
% % % % % % % login/page.tsx # Magic Link login
% % % % % % % signup/page.tsx # Cadastro com nome + email
% % % % % % % callback/route.ts # Handler OAuth / Magic Link
% % % % % % api/
% % % % % % % ai/ # Endpoints de IA
% % % % % % % user/
% % % % % % % % delete/ # DELETE para LGPD
% % % % % % % layout.tsx # Root layout (fontes, tema, LGPD banner)
% % % % % % components/
% % % % % % % ui/ # Componentes base reutilizáveis
% % % % % % % % Button.tsx
% % % % % % % % Card.tsx
% % % % % % % % Input.tsx
% % % % % % % % Textarea.tsx
% % % % % % % % FormField.tsx # Wrapper de campo com label + hint + erro
% % % % % % % % FormError.tsx # Mensagem de erro com role="alert"
% % % % % % % % FormHint.tsx # Texto auxiliar abaixo do campo
% % % % % % % % LoadingButton.tsx # Botão com spinner durante submissão
% % % % % % % % Toast.tsx # Notificação temporária
% % % % % % % % Badge.tsx
% % % % % % % % ProgressBar.tsx
% % % % % % % layout/
% % % % % % % % Sidebar.tsx # Navegação lateral
% % % % % % % % ThemeToggle.tsx # Toggle dark/light
% % % % % % modules/ # Views por feature (lógica de página)
% % % % % % % dashboard/DashboardView.tsx
% % % % % % % library/
% % % % % % % % LibraryView.tsx
% % % % % % % % AddContentModal.tsx
% % % % % % % focus/
% % % % % % % % review/ReviewView.tsx
% % % % % % % % active/ActiveView.tsx
% % % % % % % % skills/SkillsView.tsx
% % % % % % % % help/HelpView.tsx
% % % % % % engine/ # Cognitive Engine (algoritmos puros)
% % % % % % % index.ts # API pública
% % % % % % % spaced-repetition/ # SM-2 + scheduling
% % % % % % % retention/ # Ebbinghaus + forgetting risk
% % % % % % % mastery/ # Mastery score
% % % % % % % skill-evolution/ # Skill progression
% % % % % % % active-learning/ # Pirâmide de Glasser
% % % % % % % cognitive-score/ # Score global
% % % % % % services/ # Acesso ao Supabase por domínio
% % % % % % % contentsService.ts
% % % % % % % flashcardsService.ts
% % % % % % % skillsService.ts
% % % % % % % sessionsService.ts
% % % % % % % reviewService.ts
% % % % % % % retentionService.ts
% % % % % % % cognitiveEventsService.ts
% % % % % % % localStorageService.ts
% % % % % % store/
% % % % % % % % AppContext.tsx # Estado global + reducer + sync Supabase
% % % % % % % % ToastContext.tsx # Toasts (máx 3 simultâneos)
% % % % % % % % FocusSessionContext.tsx # Estado global do timer (isRunning)
% % % % % % components/
% % % % % % % % analytics/
% % % % % % % % % PostHogProvider.tsx # Init PostHog + page views + LGPD opt-in/out
% % % % % % % % % AnalyticsIdentifier.tsx # Identifica usuário no PostHog e Sentry
% % % % % % % % lgpd/
% % % % % % % % % ConsentBanner.tsx # Banner LGPD (nl_lgpd_consent)
% % % % % % hooks/
% % % % % % % % useAppData.ts
% % % % % % % % useToast.ts
% % % % % % % % useTheme.ts
% % % % % % % % usePushNotifications.ts # Permissão reativa, subscribe/unsubscribe (PWA-01)
% % % % % % % % useAnalytics.ts # Hook type-safe: track(), identifyUser(), resetUser()
% % % % % % lib/
% % % % % % % % supabase/
% % % % % % % % % client.ts # createBrowserClient
% % % % % % % % % server.ts # createServerClient (cookies)
% % % % % % % % validation/
% % % % % % % % % schemas.ts # Schemas Zod centralizados

```

```

% % % security/
% % % % % rateLimit.ts
% % % % % validation.ts
% % % % % sanitize.ts
% % % % % rbac.ts
% % % % % logger.ts
% % % % ai/
% % % % % client.ts
% % % % % prompts.ts
% % % types/
% % % % % index.ts # Tipos de domínio
% % % % % database.types.ts # Gerado pelo Supabase
% % % % test/
% % % % % setup.ts # @testing-library/jest-dom
% % % tests/
% % % % % e2e/
% % % % % % % global.setup.ts # Auth setup via Supabase Admin API
% % % % % % % pages/ # Page Objects
% % % % % % % % % LibraryPage.ts
% % % % % % % % % ReviewPage.ts
% % % % % % % % % utils/helpers.ts
% % % % % % % % % auth.spec.ts
% % % % % % % % % app.spec.ts
% % % % % % % % % ux-01-validation.spec.ts
% % % % % % % % % library.spec.ts
% % % % % % % % % review.spec.ts
% % % % % % % % % accessibility.spec.ts
% % % docs/ # Documentação oficial
% % % public/
% % % % % landing.html
% % % % % middleware.ts # Rate limit + RBAC + session guard
% % % % % next.config.ts # Security headers + withSentryConfig + CSP expandida
% % % % % sentry.server.config.ts # Sentry init – server runtime
% % % % % sentry.edge.config.ts # Sentry init – edge runtime
% % % % % src/instrumentation.ts # Next.js hook: register() + onRequestError
% % % % % src/instrumentation-client.ts # Sentry init – browser (Session Replay)
% % % % % vitest.config.ts
% % % % % playwright.config.ts
% % % % % .env.local # Nunca commitar

```

# Observabilidade

## Sentry v10 — Error Tracking

O Sentry captura automaticamente erros em todos os runtimes:

| Runtime          | Arquivo de inicialização                             |
|------------------|------------------------------------------------------|
| Browser          | src/instrumentation-client.ts                        |
| Node.js (server) | sentry.server.config.ts (via src/instrumentation.ts) |
| Edge             | sentry.edge.config.ts (via src/instrumentation.ts)   |

- Session Replay: gravações de sessão com maskAllText e blockAllMedia — 100% em erros, 10% em sessões normais
- onRequestError hook: captura erros em Server Components e Route Handlers
- global-error.tsx: tela de fallback PT-BR para erros de renderização React; reporta ao Sentry via captureException
- Source maps: upload automático no build via SENTRY\_AUTH\_TOKEN

## PostHog — Product Analytics

O PostHog rastreia comportamento de usuário respeitando o consentimento LGPD:

- LGPD: accepted !' opt\_in\_capturing() | minimal/ausente !' opt\_out\_capturing()
- Page views: rastreados manualmente via usePathname + useSearchParams (Next.js App Router)
- Identificação: AnalyticsIdentifier chama posthog.identify(userId) ao entrar na área autenticada

## Hook `useAnalytics`

```
const { track, identifyUser, resetUser } = useAnalytics()

// Eventos tipados – TypeScript garante propriedades corretas por evento
track('session_started', { content_id, phase, duration_secs })
track('achievement_unlocked', { achievement_id, xp_gained })
```

## Variáveis de Ambiente de Observabilidade

| Variável                 | Escopo  | Obrigatória                       |
|--------------------------|---------|-----------------------------------|
| NEXT_PUBLIC_SENTRY_DSN   | Público | Para error tracking               |
| SENTRY_AUTH_TOKEN        | Privado | Para source maps no build         |
| SENTRY_ORG               | Privado | Para source maps no build         |
| SENTRY_PROJECT           | Privado | Para source maps no build         |
| NEXT_PUBLIC_POSTHOG_KEY  | Público | Para analytics                    |
| NEXT_PUBLIC_POSTHOG_HOST | Público | Default: https://us.i.posthog.com |

Todas as vars devem ser adicionadas também no Vercel (Settings !' Environment Variables).

# Padrões Arquiteturais

## Server Components vs Client Components

- Server Components (padrão no App Router): layouts, páginas de carregamento, leitura de sessão
- Client Components ("use client"): qualquer componente com estado, eventos, hooks, contexto
- Regra: preferir Server Components; usar Client apenas onde necessário

## Formulários

Todos os formulários seguem o padrão:

```
const form = useForm<FormValues>({
  resolver: zodResolver(schema),
  defaultValues: { ... }
})
// novalidate no <form> obrigatório
// setFocus no primeiro campo com erro via onError
// LoadingButton com isPending
// FormField > Input + FormError + FormHint
```

## Validação

Schemas Zod centralizados em src/lib/validation/schemas.ts:

```
const EMAIL_REGEX = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/i
export const emailSchema = z.string().trim()
  .min(1, 'Email é obrigatório')
  .refine(val => EMAIL_REGEX.test(val), 'Email inválido')
```

## Gestão de Estado

- AppContext: estado global da aplicação (contents, flashcards, skills, user)
  - %æ Reducer puro exportado (appReducer + EMPTY\_STATE)
  - %æ Side effects assíncronos para sync com Supabase
- ToastContext: fila de toasts (máximo 3 simultâneos, LIFO)
- Sem Redux ou Zustand — React Context é suficiente para o escopo atual

---

## Frontend Architecture

### Roteamento

```
/                !' redireciona para /landing.html
/auth/login      !' página pública (sem auth)
/auth/signup     !' página pública (sem auth)
/auth/callback   !' handler OAuth/Magic Link
/dashboard       !' área autenticada
/library         !' área autenticada
/focus           !' área autenticada (seleção de conteúdo)
/focus/[contentId] !' área autenticada (sessão de foco)
/review          !' área autenticada
/active          !' área autenticada
/skills          !' área autenticada
/help            !' área autenticada
```

### Theming

- Toggle dark/light via data-theme attribute em <html>
  - CSS custom properties em globals.css
-

- Script inline no <head> para prevenir flash de tema (FOUC)
- useTheme() hook expõe theme e toggleTheme()

### Componentes UI Base

Todos em src/components/ui/:

| Componente    | Props principais                | Uso                            |
|---------------|---------------------------------|--------------------------------|
| FormField     | label, required, hint, children | Wrapper de campo de formulário |
| FormError     | id, message                     | Erro com role="alert"          |
| FormHint      | id, children                    | Texto auxiliar                 |
| LoadingButton | isLoading, children             | Botão com spinner              |
| Toast         | via ToastContext                | Notificação temporária         |
| Button        | variant, size                   | Botão base                     |
| Card          | className                       | Container visual               |
| Input         | extends HTMLInput               | Input base                     |
| Textarea      | extends HTMLTextarea            | Textarea base                  |

## Backend Architecture

### Next.js API Routes

Localizadas em src/app/api/:

| Rota             | Método | Descrição                               |
|------------------|--------|-----------------------------------------|
| /api/user/delete | DELETE | Exclusão de dados do usuário (LGPD)     |
| /api/ai/*        | POST   | Endpoints de IA (geração de flashcards) |

### Middleware (`middleware.ts`)

Executado em todas as rotas (exceto \_next/static, imagens, favicon):

1. Session guard: rota de app sem sessão !' redirect /auth/login
2. Auth redirect: rota de auth com sessão !' redirect /dashboard
3. Rate limiting: 5 req/15min por IP em /auth/\*
4. RBAC: bloqueia /api/admin/\* para roles insuficientes
5. Security logging: registra eventos de segurança em JSON

### Clientes Supabase

```
// Browser (Client Components)
import { createClient } from '@lib/supabase/client'

// Server (Server Components, API Routes, Middleware)
import { createClient } from '@lib/supabase/server'
```

Regra crítica: SERVICE\_ROLE\_KEY apenas no servidor (Node.js). Nunca expor ao browser.



# Banco de Dados

## Diagrama de Tabelas

```
users
  id (uuid, FK !' auth.users)
  email, name, role, total_xp, streak, last_study_date

contents
  id, user_id, title, type, author, description
  ContentType: 'book' | 'course' | 'video' | 'article' | 'podcast' | 'other'

flashcards
  id, content_id, user_id
  front, back (texto da pergunta/resposta)
  ef (ease factor), interval, reps, nextReview
  mastery: 'new' | 'learning' | 'review' | 'strong'

skills
  id, name, category, description (habilidades globais)

user_skills
  user_id, skill_id, xp, level (relação usuário !" habilidade)

study_sessions
  id, user_id, content_id
  duration, highlights_count, notes, completed_at

review_cycles
  id, user_id, flashcard_id
  quality (1-5), response_time_ms, retention_before, retention_after
  reviewed_at

retention_snapshots
  id, user_id, flashcard_id
  retention_score, risk_level, snapshot_date

cognitive_events
  id, user_id, event_type, metadata, created_at
```

## RLS (Row Level Security)

Todas as tabelas têm RLS ativo. Política padrão:

```
USING (auth.uid() = user_id)
WITH CHECK (auth.uid() = user_id)
```

## Tipos de Domínio (`src/types/index.ts`)

```
type ContentType = 'book' | 'course' | 'video' | 'article' | 'podcast' | 'other'
type CardMastery = 'new' | 'learning' | 'review' | 'strong'
type SkillCategory = 'technical' | 'soft' | 'cognitive' | 'creative' | 'other'
type UserRole = 'user' | 'admin' | 'super_admin'
```

## Cognitive Engine

O Cognitive Engine é uma biblioteca pura de algoritmos TypeScript em `src/engine/`. Nenhum módulo importa do Supabase ou do Next.js — são funções testáveis em isolamento.

### API Pública (`src/engine/index.ts`)

```
// SM-2 aprimorado
export { calculateSM2 } from './spaced-repetition/sm2'
export { buildReviewQueue } from './spaced-repetition/scheduling'

// Retenção
export { calcRetention, calcStability } from './retention/retentionModel'
export { calcRiskLevel } from './retention/forgettingRisk'

// Mastery
export { calcCardMastery, calcContentMastery } from './mastery/masteryScore'

// Skills
export { calcSkillProgression } from './skill-evolution/skillProgression'

// Active Learning (Pirâmide de Glasser)
export { calcActiveLearningScore } from './active-learning/activeLearningScore'

// Score cognitivo global
export { calcCognitiveScore } from './cognitive-score/cognitiveScore'
```

### SM-2 Aprimorado

Baseado no algoritmo SuperMemo 2 com bônus por tempo de resposta rápido:

```
EF_new = max(1.3, EF_old + 0.1 * (5^q) * (0.08 + (5^q) * 0.02))
bonus = responseTimeMs < 5000 ? +0.05 : 0
nextInterval = q < 3 ? 1 : reps === 1 ? 1 : reps === 2 ? 6 : round(interval * EF)
```

### Modelo de Retenção (Ebbinghaus)

```
R(t) = 100 * e^(-t/S)
S = intervalDays * easeFactor  (estabilidade da memória)
```

### Risco de Esquecimento

```
high    !' R < 40% OU nextReview vencido
medium  !' R < 65% OU nextReview em < 1 dia
low     !' demais casos
```

### Fila de Revisão

1. Filtrar flashcards com `nextReview <= agora`
2. Ordenar: `high !' medium !' low`
3. Desempate: `nextReview` crescente

### Mastery Score (0–100)

```
base: new=0, learning=25, review=50, strong=75
score = base × min(1, retention/100)
contentMastery = média dos cards do conteúdo
```

## Cognitive Score Global (0–100)

```
score = retention × 0.35
      + mastery   × 0.30
      + consistency × 0.20
      + activeLearning × 0.15
```

## Sistema de Revisão Espaçada

### Fluxo Completo

```
1. Usuário abre /review
2. ReviewView chama buildReviewQueue(flashcards)
3. Cards due são apresentados um a um
4. Usuário avalia: 1 (blackout) !' 5 (perfeito)
5. calcSM2() recalcula EF, interval, nextReview
6. reviewService.recordReviewCycle() persiste no banco
7. AppContext dispara UPDATE_CONTENT_PROGRESS
8. Fila atualizada no estado local
```

### Qualidades de Avaliação (escala 0-5)

| Qualidade | Significado                                       |
|-----------|---------------------------------------------------|
| 0–2       | Falha total / confusão — reinicia interval para 1 |
| 3         | Correto com dificuldade                           |
| 4         | Correto com hesitação                             |
| 5         | Correto e imediato                                |

## Sistema de Autenticação

### Fluxo Magic Link

```
1. Usuário informa email em /auth/login ou /auth/signup
2. Supabase envia email com link único (1 uso, expira em 1h)
3. Usuário clica no link !' redirect para /auth/callback
4. callback/route.ts troca code por session
5. Redirect para /dashboard
6. Middleware valida session em cada requisição
```

### Proteção de Rotas

```
// middleware.ts – simplificado
if (isAppRoute && !session) return redirect('/auth/login')
if (isAuthRoute && session) return redirect('/dashboard')
```

## Rate Limiting em Auth

- 5 requisições por IP em 15 minutos para /auth/\*
- Retorna HTTP 429 com Retry-After header
- Implementado em memória (edge-compatible)

## Segurança

### HTTP Security Headers (`next.config.ts`)

| Header                    | Valor                                          |
|---------------------------|------------------------------------------------|
| Content-Security-Policy   | default-src 'self' + exceções necessárias      |
| Strict-Transport-Security | max-age=31536000; includeSubDomains (produção) |
| X-Frame-Options           | DENY                                           |
| X-Content-Type-Options    | nosniff                                        |
| Referrer-Policy           | strict-origin-when-cross-origin                |
| Permissions-Policy        | camera, microphone, geolocation desabilitados  |

### Sanitização (`src/lib/security/sanitize.ts`)

```
stripHtml(input)           // Remove todas as tags HTML
escapeHtml(input)          // Escapa <, >, &, ", '
sanitizeFileName(name)     // Remove caracteres perigosos de nomes de arquivo
isValidMimeType(type)      // Whitelist de MIME types aceitos
```

### RBAC (`src/lib/security/rbac.ts`)

```
hasRole(userRole, required) // Verifica se role atende ao mínimo
isAdmin(role)               // admin | super_admin
isSuperAdmin(role)          // super_admin apenas
requireRole(role, required) // Lança erro se insuficiente
```

## LGPD

- Banner de consentimento em src/components/lgpd/ConsentBanner.tsx
- localStorage para persistir preferências
- DELETE /api/user/delete remove dados de todas as tabelas + auth.admin.deleteUser

## Progressive Web App (PWA)

### Visão Geral

O NeuroLearn é instalável como PWA em Android, iOS (Safari) e desktop (Chrome/Edge). A implementação usa Service Worker nativo sem next-pwa para evitar conflito com o webpack do

@sentry/nextjs.

Manifesto (public/manifest.webmanifest)

```
{
  "name": "NeuroLearn",
  "short_name": "NeuroLearn",
  "start_url": "/dashboard",
  "display": "standalone",
  "background_color": "#0a0b0f",
  "theme_color": "#7c3aed",
  "icons": [
    { "src": "/icons/icon-192.png", "sizes": "192x192", "purpose": "any" },
    { "src": "/icons/icon-512.png", "sizes": "512x512", "purpose": "any" },
    { "src": "/icons/icon-maskable.png", "sizes": "512x512", "purpose": "maskable" }
  ],
  "shortcuts": [
    { "name": "Revisão diária", "url": "/review" },
    { "name": "Biblioteca", "url": "/library" }
  ]
}
```

Ícones gerados via scripts/generate-icons.js (Node.js puro, sem dependências externas — usa zlib + CRC32 para criar PNGs válidos).

Service Worker (public/sw.js)

Estratégias de cache por tipo de recurso:

| Recurso        | Estratégia                                    |
|----------------|-----------------------------------------------|
| /_next/static/ | Cache First — ativos imutáveis com hash       |
| Páginas HTML   | Network First !' fallback do cache            |
| /api/*         | Network Only — nunca cachear respostas de API |
| Push events    | showNotification com actions Revisar/Depois   |

O SW usa skipWaiting() no install e clients.claim() no activate para ativação imediata.

Push Notifications

Infraestrutura

- Biblioteca: web-push (server-side, Node.js)
- Protocolo: Web Push Protocol com VAPID
- Tabela: push\_subscriptions no Supabase (RLS: auth.uid() = user\_id)

Tabela push\_subscriptions

```
CREATE TABLE push_subscriptions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  endpoint TEXT NOT NULL,
  p256dh TEXT NOT NULL,
  auth TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (user_id, endpoint)
);
ALTER TABLE push_subscriptions ENABLE ROW LEVEL SECURITY;
CREATE POLICY "users can manage own push subscriptions"
  ON push_subscriptions FOR ALL
  USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);
```

## API Routes

| Rota                | Método | Descrição                                                                        |
|---------------------|--------|----------------------------------------------------------------------------------|
| /api/push/subscribe | POST   | Salva subscription via upsert<br>onConflict: user_id.endpoint                    |
| /api/push/subscribe | DELETE | Remove subscription por endpoint do usuário autenticado                          |
| /api/push/notify    | POST   | Envia push para todos os usuários com cards vencidos (protegido por CRON_SECRET) |

## Fluxo de envio (cron diário)

1. Buscar flashcards com next\_review <= hoje
2. Agrupar por user\_id (contagem)
3. Buscar push\_subscriptions dos usuários afetados
4. Enviar via webpush.sendNotification() para cada endpoint
5. Remover subscriptions com HTTP 410 (expiradas)

## Variáveis de Ambiente

| Variável                     | Exposição         | Descrição                                                              |
|------------------------------|-------------------|------------------------------------------------------------------------|
| NEXT_PUBLIC_VAPID_PUBLIC_KEY | Pública (browser) | Chave VAPID pública — usada no <code>pushManager.subscribe()</code>    |
| VAPID_PRIVATE_KEY            | Servidor only     | Chave VAPID privada — usada no <code>web-push.setVapidDetails()</code> |
| VAPID_EMAIL                  | Servidor only     | Email de contato para o protocolo VAPID                                |
| CRON_SECRET                  | Servidor only     | Token para proteger POST /api/push/notify                              |

## Hook `usePushNotifications`

```
// src/hooks/usePushNotifications.ts
export function usePushNotifications(): {
  permission: 'default' | 'granted' | 'denied' | 'unsupported'
  isSubscribed: boolean
  loading: boolean
  subscribe(): Promise<boolean>
  unsubscribe(): Promise<void>
}
```

## Componentes

- **ServiceWorkerRegistrar** — Client Component que registra /sw.js uma única vez após mount. Renderiza null.
- **PushNotificationPrompt** — Banner fixo (bottom-center) que aparece 3s após mount se: permissão não negada, não subscrito, não dispensado (localStorage nl\_push\_dismissed).

Ambos adicionados ao AppShell.tsx após <AnalyticsIdentifier />.

## Geração de Ícones

```
node scripts/generate-icons.js
```

Gera os 4 ícones PNG em public/icons/ usando apenas módulos nativos do Node.js (zlib, fs, path). Não é necessário rodar em CI — os ícones estão commitados.

## Estratégia de Testes

### Unitários (Vitest)

Configuração (vitest.config.ts):

- Ambiente padrão: node
- Arquivos DOM: // @vitest-environment jsdom no topo do arquivo
- globals: true para matchers do jest-dom
- plugins: [react()] para transpilação JSX
- setupFiles: ['src/test/setup.ts']

Padrão de mock para Supabase:

```
const mockChain = { from: vi.fn(), select: vi.fn(), eq: vi.fn(), ... }
Object.values(mockChain).forEach(fn => (fn as Mock).mockReturnValue(mockChain))
vi.mock('@lib/supabase/client', () => ({ createClient: () => mockChain }))
// Override por teste:
mockChain.select.mockResolvedValueOnce({ data: [...], error: null })
// Import dinâmico após mock:
const { listContents } = await import('./contentsService')
```

### E2E (Playwright)

Projetos:

```
setup          !' global.setup.ts (Supabase Admin API !' storageState)
chromium       !' testes sem auth (ignora library/review/accessibility)
authenticated !' storageState: tests/e2e/.auth/user.json
```

Auth setup:

```
// global.setup.ts
const { data } = await supabase.auth.admin.generateLink({
  type: 'magiclink',
  email: process.env.TEST_USER_EMAIL!
})
await page.goto(data.properties.action_link)
await page.context().storageState({ path: AUTH_FILE })
```

Variáveis de ambiente para E2E:

- NEXT\_PUBLIC\_SUPABASE\_URL
- SUPABASE\_SERVICE\_ROLE\_KEY (somente Node.js, nunca browser)

- TEST\_USER\_EMAIL

### Comandos de Teste

```
npm run test:unit      # Vitest (260 testes)
npm run test:e2e       # Playwright (multi-project)
npm run test:coverage  # Cobertura com thresholds
```

Thresholds de cobertura:

| Métrica    | Mínimo |
|------------|--------|
| Statements | 80%    |
| Branches   | 70%    |
| Functions  | 80%    |
| Lines      | 80%    |

## Estratégia de IA

### Configuração

```
// src/lib/ai/client.ts
import OpenAI from 'openai'
export const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY })

import Anthropic from '@anthropic-ai/sdk'
export const anthropic = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY })
```

### Prompts Estruturados (`src/lib/ai/prompts.ts`)

Prompts centralizados para:

- buildFlashcardPrompt — geração de flashcards a partir de conteúdo e highlights
- buildTeachingPrompt — análise da explicação do usuário no Modo Professor
- buildCoachPrompt — geração de recomendações personalizadas pelo Cognitive Coach
- buildQuizPrompt — geração de distratores plausíveis para o Quiz Adaptativo

### Endpoints de IA

| Rota                             | Entrada                         | Saída                                                       |
|----------------------------------|---------------------------------|-------------------------------------------------------------|
| POST /api/ai/generate-flashcards | notes, highlights, title, count | Array de { front, back }                                    |
| POST /api/ai/analyze-teaching    | teachText, topic                | TeachAnalysis (clarity_score, coverage_score and strengths) |
| POST /api/ai/cognitive-coach     | métricas cognitivas             | Mensagem de coaching personalizada                          |
| POST /api/ai/generate-quiz       | front, back, count              | { distractors: string[] }                                   |

Todos os schemas de validação estão centralizados em src/lib/ai/validation.ts (Zod).

### Quiz Adaptativo (`/active` — modo Auto-Avaliação)



```
1. Usuário seleciona modo "0>Yé Auto-Avaliação" e escolhe um conteúdo
2. loadQuiz() filtra flashcards do conteúdo, ordena por mastery (new !' learning !' review !' strong), limita a 7
3. Promise.allSettled() chama /api/ai/generate-quiz em paralelo para cada card
4. Fisher-Yates embaralha [correct_answer + 3 distractors] para cada pergunta
5. Usuário responde !' feedback imediato (verde/vermelho) !' Próxima
6. Ao fim: placar + XP (10 por acerto) + lista de cards para revisar
```

Cards com mastery = 'new' aparecem primeiro para priorizar consolidação de pontos fracos.

## Análise do Modo Professor (`/active` — modo Modo Professor)

```
1. Usuário digita explicação "e 100 chars
2. Botão "Analisar com IA" fica visível
3. analyzeTeaching() POST /api/ai/analyze-teaching
4. Paine! TeachAnalysis exibe: clarity_score, coverage_score, gaps, strengths, suggestions, estimated_retention
```

analyzingRef (useRef síncrono) impede double-submit mesmo com React 19 concurrent mode.

## Deploy e Variáveis de Ambiente

### Variáveis obrigatórias (.env.local — nunca commitar)

```
NEXT_PUBLIC_SUPABASE_URL=https://<project-id>.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGci...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGci... # APENAS servidor
ANTHROPIC_API_KEY=sk-ant-api03-...
OPENAI_API_KEY=sk-proj-...
```

### Variáveis para testes E2E

```
TEST_USER_EMAIL=test@example.com
```

### Configuração Supabase (Dashboard)

- Site URL: http://localhost:3003 (dev) / URL de produção
- Redirect URLs: http://localhost:3003/auth/callback
- Email Auth: habilitado
- Magic Link: habilitado

### Build

```
npm run build # Next.js production build
# Nunca rodar com o servidor dev ativo na mesma pasta
# Se ocorrer erro 500 "Cannot find module './NNN.js' !" npm run dev:clean
```

## Convenções de Código

### TypeScript

- strict: true em tsconfig.json
- Sem any (use unknown + type guards)

- Interfaces para objetos de dados; type para unions e aliases
- Imports com alias @/ !' src/

## Nomenclatura

| Tipo              | Convenção                  | Exemplo                  |
|-------------------|----------------------------|--------------------------|
| Componentes       | PascalCase                 | FormField.tsx            |
| Hooks             | camelCase + use prefix     | useTheme.ts              |
| Services          | camelCase + Service suffix | contentsService.ts       |
| Types             | PascalCase                 | ContentType, CardMastery |
| Constants         | UPPER_SNAKE_CASE           | MAX_TOASTS, EMAIL_REGEX  |
| Arquivos de teste | .test.ts / .test.tsx       | sm2.test.ts              |

## Commits (padrão descritivo)

```
feat: adicionar FormField e FormError para UX-01
fix: corrigir aria-describedby no AddContentModal
test: adicionar testes unitários para contentsService
docs: atualizar TESTING.md com padrão de mock Supabase
```

## Comentários

- Apenas onde o "porquê" é não-óbvio
- Sem comentários que explicam "o quê" (o código já faz isso)
- Documentação em português

## Performance

### Frontend

- Server Components por padrão (zero bundle JS para lógica server)
- Lazy loading de Client Components com dynamic() quando adequado
- CSS Tailwind purged no build (apenas classes usadas)
- Fontes via next/font (sem layout shift)

### Banco de Dados

- Queries paralelas no AppContext mount (4 queries simultâneas via Promise.all)
- getSession() em vez de getUser() no layout (elimina round-trip de verificação)
- Índices nas colunas user\_id e nextReview (colunas de filtro principal)

### Cognitive Engine

- Algoritmos síncronos e puros (zero latência de I/O)
- buildReviewQueue roda em memória (sem queries adicionais)

## Troubleshooting

### Erro 500 "Cannot find module './NNN.js'"

Causa: Build realizado com servidor dev ativo — chunks stale em cache.

Solução:

```
# Parar o servidor dev, depois:  
npm run dev:clean
```

### Magic Link não chega / Rate limit

Causa: Free tier Supabase limita envios de email.

Solução: Aguardar ~1h ou configurar SMTP customizado no dashboard Supabase.

### Testes E2E falhando por auth

Causa: tests/e2e/.auth/user.json expirado.

Solução: Deletar o arquivo e rodar `npm run test:e2e` novamente (`global.setup.ts` regenera).

### `expect is not defined` no Vitest

Causa: `globals: true` ausente no `vitest.config.ts` ou `setup.ts` não importa `jest-dom`.

Solução: Verificar `globals: true` e `import '@testing-library/jest-dom'` em `src/test/setup.ts`.

### IDE mostrando erros TypeScript após edição

Causa: Cache de diagnósticos do IDE (stale).

Solução: Verificar o estado real com `npx tsc --noEmit`.

### Supabase RLS bloqueando query

Causa: Query usando cliente browser mas sem sessão ativa.

Solução: Usar cliente server ou verificar `auth.uid()` na policy.

---

## Gate de Qualidade

Antes de qualquer entrega:

```
npm run type-check      # Zero erros TypeScript  
npm run lint            # Zero warnings ESLint  
npm run test:unit       # 260+ testes passando  
npm run build           # Build limpo
```

Após qualquer feature com UI:

```
npm run test:e2e        # Playwright (todos os projetos)
```

---





















































































